

FPGA-Based Handwritten Character Recognition Accelerator Using a Quantized CNN on the PYNQ-Z2

Satyarajsinh Gohil

B.Tech., Electronics and VLSI Design

Dhirubhai Ambani University (DAU), Gandhinagar, Gujarat, India

Abstract—Convolutional neural networks (CNNs) achieve high accuracy in handwritten character recognition but their deployment on embedded platforms is constrained by computational complexity, memory bandwidth, and power. This paper presents the design, verification, and physical validation of a complete 62-class handwritten character recognition accelerator implemented in custom Verilog RTL on the PYNQ-Z2 platform (Xilinx Zynq-7020, XC7Z020CLG400-1). A CNN comprising three convolutional layers, two max-pooling layers, a 128-neuron dense layer, and a 62-class output layer was trained on the EMNIST ByClass dataset using TensorFlow/Keras, achieving 82.7893% floating-point test accuracy over 116,323 test samples. Batch-normalization parameters were folded into the convolutional kernels and the inference model was quantized to signed 16-bit Q8.8 fixed-point format. A key implementation challenge was activation-memory inference: an initial asynchronous read architecture caused Vivado to infer approximately 27,174 distributed LUT-based memory elements against a budget of roughly 17,400, preventing implementation. Redesigning the memories around synchronous accesses enabled block RAM inference and reduced LUT utilization to 7.38%. The RTL was verified layer-wise against software golden-reference data with zero mismatches across all seven checkpoints. The final implementation consumes 7.38% LUT, 2.33% flip-flop, 78.21% BRAM, and 3.64% DSP resources, and achieves timing closure at 50 MHz with a worst negative slack of +1.545 ns. The bitstream was deployed on physical hardware and validated through an AXI GPIO host interface, returning the expected class, checksum, and status values.

Index Terms—FPGA, CNN accelerator, handwritten character recognition, EMNIST, PYNQ-Z2, Zynq-7020, Verilog, fixed-point arithmetic, Q8.8, BRAM, hardware acceleration.

I. INTRODUCTION

Handwritten character recognition is a fundamental problem in computer vision, underpinning document digitization, postal automation, cheque processing, and optical character recognition. Classical approaches relied on hand-engineered features followed by a statistical classifier. Convolutional neural networks have largely displaced these methods by learning hierarchical spatial features directly from pixel data [1], [2].

CNN inference, however, is dominated by multiply-accumulate (MAC) operations and by frequent access to weights and intermediate feature maps. On resource-constrained embedded platforms these two characteristics translate directly into area, bandwidth, and power pressure. Field-programmable gate arrays are attractive targets because dedicated datapaths can be synthesized for convolution, activation, pooling, and fully connected stages, and because

on-chip block RAM provides deterministic, low-latency local storage for parameters and activations [3], [4].

This work implements a complete CNN inference accelerator on the PYNQ-Z2, which pairs Zynq-7020 programmable logic with an ARM processing system, allowing Python host software to drive custom hardware. The development flow spans software training, batch-normalization folding, fixed-point quantization, RTL design, simulation verification, synthesis and implementation, and physical board validation.

The principal contributions are: (i) a 62-class CNN partitioned for FPGA deployment; (ii) conversion of the trained network into a fixed-point, inference-only model; (iii) custom Verilog implementations of the convolution, ReLU, max-pooling, dense, and argmax stages; (iv) a BRAM-oriented activation-memory architecture that resolves a distributed-memory over-utilization failure; (v) layer-wise RTL verification against golden references; and (vi) integration with the Zynq processing system over AXI GPIO with successful hardware validation.

II. SYSTEM OVERVIEW

The system comprises four stages: dataset preparation and CNN training; model conversion and fixed-point quantization; FPGA RTL implementation; and PYNQ-Z2 deployment and validation. The flow proceeds from the EMNIST ByClass dataset through TensorFlow/Keras training, batch-normalization folding and model conversion, Q8.8 quantization, Verilog RTL design, Vivado synthesis and implementation, and finally on-board inference producing a predicted character class.

Host software executing on the Zynq processing system configures the accelerator, streams the input image, and reads back the classification result together with diagnostic status. All inference computation is performed in programmable logic.

III. DATASET AND PREPROCESSING

The EMNIST ByClass dataset is used, containing 62 classes spanning digits 0–9, uppercase letters A–Z, and lowercase letters a–z. The original partition provides 697,932 training and 116,323 test samples; ten percent of the training set was held out for validation, giving 69,793 validation and 628,139 training samples. Table I summarizes the configuration.

TABLE I
DATASET CONFIGURATION

Parameter	Value
Dataset	EMNIST ByClass
Number of classes	62
Original training samples	697,932

Validation samples	69,793
Final training samples	628,139
Test samples	116,323
Original image size	28 × 28
FPGA input size	32 × 32 × 1
Pixel representation	Grayscale

Preprocessing consists of orientation correction by transposition, bilinear resizing from 28×28 to 32×32 , and normalization of pixel intensities to the range $[0, 1]$. The resulting $32 \times 32 \times 1$ tensor is subsequently converted to the fixed-point representation used by the hardware datapath.

IV. CNN ARCHITECTURE

The network uses three convolutional layers with valid (unpadded) 3×3 kernels, two 2×2 max-pooling layers with stride 2, a 128-neuron fully connected hidden layer, and a 62-class output layer followed by argmax. ReLU activation follows every convolutional layer and the hidden dense layer; the output layer produces raw signed scores. Layer dimensions are given in Table II.

TABLE II
CNN ARCHITECTURE AND LAYER DIMENSIONS

Layer	Configuration	Output
Input	Grayscale	$32 \times 32 \times 1$
Conv1	3×3 , 16 filters	$30 \times 30 \times 16$
Pool1	2×2 , stride 2	$15 \times 15 \times 16$
Conv2	3×3 , 32 filters	$13 \times 13 \times 32$
Pool2	2×2 , stride 2	$6 \times 6 \times 32$
Conv3	3×3 , 64 filters	$4 \times 4 \times 64$
Flatten	—	1024
Dense1	128 neurons, ReLU	128
Output	62 classes	62

V. SOFTWARE TRAINING

The model was trained in TensorFlow/Keras using the Adam optimizer with an initial learning rate of 0.001, a batch size of 128, and a maximum of 40 epochs. Learning-rate reduction on plateau used a factor of 0.5 with a patience of four epochs and a floor of 10^{-6} . Early stopping used a patience of twelve epochs with best-weight restoration and checkpointing. Class weighting was enabled to compensate for the pronounced class imbalance of EMNIST ByClass; data augmentation was not used. A fixed shuffle seed of 42 was applied for reproducibility. Batch normalization followed each convolutional layer and dropout with rate 0.3 preceded the dense classifier.

VI. FPGA MODEL CONVERSION

A. Batch-Normalization Folding

Batch normalization is required during training but represents unnecessary hardware in the inference datapath. Its affine transformation is therefore folded into the preceding convolution. Defining the scale factor

$$s = \gamma / \sqrt{(\sigma^2 + \epsilon)} \quad (1)$$

where γ is the learned scale, σ^2 the running variance, and ϵ a numerical-stability constant, the folded kernel and bias are

$$W' = W \cdot s \quad (2)$$

$$b' = \beta - \mu s \quad (3)$$

with μ and β the running mean and learned offset. This removes the batch-normalization datapath from the FPGA entirely at no accuracy cost.

B. Inference Model

The deployed inference graph consists of Conv1–ReLU–Pool1, Conv2–ReLU–Pool2, Conv3–ReLU, flatten, Dense1–ReLU, the 62-class output layer, and argmax. Training-only operations such as dropout are removed.

VII. FIXED-POINT QUANTIZATION

Floating-point arithmetic is unnecessary for this workload and costly in programmable logic. The datapath therefore uses signed 16-bit Q8.8 fixed point, comprising eight integer and eight fractional bits. Quantization and reconstruction follow

$$x_Q = \text{round}(x \times 2^8), \quad x = x_Q / 2^8 \quad (4)$$

The product of two Q8.8 operands occupies a Q16.16 format and is rescaled by an arithmetic right shift of eight bits, preserving the sign:

$$P_{Q8.8} = (A \times B) \ggg 8 \quad (5)$$

All multiplications in the convolution and dense engines use this scaling convention. Accumulation is performed in a wider register to prevent intermediate overflow, and the result is saturated to the signed 16-bit range before being written back to activation memory.

VIII. HARDWARE ARCHITECTURE

The accelerator is implemented as hand-written Verilog RTL. The principal modules are `cnn_top` (sequencing controller), `conv_engine_q88`, `dense_mac_q88`, `mac_q88`, `maxpool_q88`, `relu_q88`, `Bram_weight_ctrl`, and `cnn_gpio_wrapper`.

A. Convolution Engine

Convolution dominates the computational load. Each output element is computed as

$$Y = \sum X_i W_i + B, \quad i = 0 \dots N-1 \quad (6)$$

where N is the number of MAC operations required for that output. Conv1 applies a 3×3 kernel over a single input channel, while Conv2 and Conv3 accumulate across 16 and 32 input channels respectively. The engine performs sequential MAC operations and reuses a single datapath rather than spatially unrolling every multiplication, which is the primary reason DSP utilization remains at 3.64%.

B. Max-Pooling

Both pooling layers use non-overlapping 2×2 windows with stride 2, computing $Y = \max(X_0, X_1, X_2, X_3)$ by sequentially fetching the four window elements. Pool1 reduces $30 \times 30 \times 16$ to $15 \times 15 \times 16$ and Pool2 reduces $13 \times 13 \times 32$ to $6 \times 6 \times 32$.

C. Dense Classifier and Argmax

The $4 \times 4 \times 64$ output of Conv3 is flattened to a 1024-element vector. Each of the 128 hidden neurons computes

$$Y_j = \sum X_i W_{ij} + B_j, \quad i = 0 \dots 1023 \quad (7)$$

followed by $\text{ReLU}(x) = \max(0, x)$. The 62-class output layer omits ReLU, and the predicted class is obtained as $c = \text{argmax}_i Y_i$, evaluated on-chip.

IX. MEMORY ARCHITECTURE AND OPTIMIZATION

Memory inference proved to be the most significant implementation obstacle, and ultimately mattered more than the arithmetic architecture.

A. Initial Architecture and Failure

The first implementation used asynchronous array reads for the large activation buffers. Although functionally correct in simulation, this coding style caused Vivado to infer distributed LUT-based memory rather than dedicated block RAM. The resulting design required approximately 27,174 LUT-based memory elements against an available budget of roughly 17,400, and implementation could not complete.

B. Synchronous Redesign

The activation memories were restructured around registered, synchronous read operations matching the block RAM inference template. This allowed the input image buffer, all intermediate activation buffers, and the weight and bias arrays to map onto dedicated BRAM primitives. LUT utilization fell to 7.38% while BRAM utilization rose to 78.21%, trading an over-subscribed resource for an under-utilized one and making the full accelerator feasible on the Zynq-7020.

X. ZYNQ INTEGRATION AND HOST HANDSHAKE

The accelerator is connected to the ARM processing system through an AXI SmartConnect driving two AXI GPIO instances, which in turn drive the CNN GPIO wrapper. The control interface exposes reset, start, pixel-valid, and pixel-data signals; the status interface returns result-valid, predicted class, a debug status field, and a 12-bit input checksum, packed into a single diagnostic word with class in bits [23:18], debug status in bits [17:12], and checksum in bits [11:0].

Hardware bring-up exposed a handshake defect invisible to RTL simulation. The host held pixel-valid asserted across multiple accelerator clock cycles, and because the input loader accepted a pixel whenever the signal was high, individual pixels were latched repeatedly. The fault was diagnosed using the checksum: for a uniform test pixel value of 5, the expected accumulation of $1024 \times 5 = 5120$ wrapped modulo 4096 to 1024, confirming repeated acceptance. Rising-edge detection was added,

$$\text{pixel_valid_rise} = \text{pixel_valid} \wedge \neg \text{pixel_valid_d} \quad (8)$$

so that only a transition is interpreted as a new pixel, guaranteeing each of the 1024 input pixels is accepted exactly once.

XI. RTL VERIFICATION

Verification compared RTL activations against golden-reference data exported from the software model at seven checkpoints rather than only at the final classification, since a correct class index can mask numerical or addressing errors upstream. Results are given in Table III.

TABLE III
LAYER-WISE RTL VERIFICATION RESULTS

CNN Stage	Values Checked	Mismatches
Conv1	14,400	0
Pool1	3,600	0
Conv2	5,408	0
Pool2	1,152	0
Conv3	1,024	0
Dense1	128	0
Output	62	0
Total	25,774	0

All seven checkpoints passed with zero mismatches across 25,774 compared values. The golden reference produced predicted class 8 with an output score of $-26,662$, an input checksum of 0xCEA, and result-valid asserted.

XII. RESULTS

A. Software Model Accuracy

The floating-point model was evaluated over the complete EMNIST ByClass test set, as summarized in Table IV.

TABLE IV
FLOATING-POINT SOFTWARE MODEL PERFORMANCE

Metric	Result
Dataset	EMNIST ByClass
Test samples	116,323
Number of classes	62
Input shape	$32 \times 32 \times 1$
Test loss	0.427682
Test accuracy	82.7893%

This figure characterizes the floating-point software model and must not be interpreted as measured FPGA accuracy; a full-dataset hardware accuracy sweep was not performed.

B. Resource Utilization

Post-implementation utilization for the XC7Z020CLG400-1 device is given in Table V.

TABLE V
FPGA RESOURCE UTILIZATION

Resource	Utilization
LUT	7.38%
Flip-Flop	2.33%
BRAM	78.21%
DSP	3.64%
BUFCTRL	3.13%

The distribution reflects the deliberate migration of parameter and activation storage from distributed logic into dedicated block RAM, and the sequential-MAC computation strategy that keeps logic and DSP demand low.

C. Timing Closure

The design was constrained to a 50 MHz target, a 20 ns clock period. Table VI reports the final static timing results.

TABLE VI
POST-IMPLEMENTATION TIMING RESULTS

Timing Metric	Result
Target frequency	50 MHz

Clock period	20 ns
Worst negative slack (WNS)	+1.545 ns
Total negative slack (TNS)	0 ns
Worst hold slack (WHS)	+0.064 ns
Total hold slack (THS)	0 ns
Pulse width slack	+8.750 ns
Failing setup endpoints	0
Failing hold endpoints	0
Failing pulse-width endpoints	0
Timing status	MET

Positive worst negative slack with zero total negative slack and no failing endpoints of any category confirms that setup, hold, and pulse-width constraints are all satisfied at 50 MHz.

XIII. PHYSICAL HARDWARE VALIDATION

The generated bitstream was deployed on a physical PYNQ-Z2 board. The Python host application loads the overlay, configures the AXI GPIO interfaces, resets the accelerator, streams the 1024 input pixels, and polls the status registers. The observed result was result-valid = 1, class = 8, debug status = 10, checksum = 0xCEA, and diagnostic word = 0x20ACEA, matching the simulation golden reference exactly.

These values remained stable across repeated reads, confirming correct end-to-end operation of the host → AXI GPIO → wrapper → CNN datapath → status readback path.

XIV. DISCUSSION AND LIMITATIONS

Four observations follow from this implementation. First, numerical representation directly governs hardware cost: Q8.8 arithmetic eliminates floating-point resources while retaining sufficient dynamic range for this network. Second, memory architecture can dominate computational architecture in FPGA CNN design — the initial design failed implementation not because of arithmetic cost but because of a memory coding style. Third, sequential MAC reuse is an effective resource-efficiency strategy where throughput requirements are modest. Fourth, layer-wise verification and an inexpensive checksum mechanism together caught both numerical and system-level integration faults that end-to-end class comparison alone would have hidden.

Limitations remain. FPGA accuracy was validated on individual samples rather than the full test set, so quantization-induced loss is unquantified. Throughput is bounded by the sequential MAC datapath, BRAM utilization at 78.21% leaves little headroom for scaling, and the AXI GPIO interface imposes per-pixel transfer overhead.

XV. FUTURE WORK

Priority extensions include a full-dataset hardware accuracy evaluation; parallel MAC instantiation to raise throughput; replacement of the GPIO transfer path with AXI-Stream or DMA; memory tiling to reduce BRAM pressure; further pipelining to exploit the available +1.545 ns margin; and power and energy-per-inference measurement.

XVI. CONCLUSION

A complete 62-class handwritten character recognition CNN accelerator was designed, verified, implemented, and

physically validated on the PYNQ-Z2 platform. Batch normalization was folded into the convolutional parameters and the network was quantized to signed Q8.8 fixed point for hardware deployment in custom Verilog RTL. Redesigning the activation memories for synchronous, BRAM-compatible access resolved a distributed-memory over-utilization failure and reduced LUT utilization to 7.38%. The RTL matched software golden references with zero mismatches across all seven layer checkpoints and 25,774 compared values. The final implementation consumes 7.38% LUT, 2.33% flip-flop, 78.21% BRAM, and 3.64% DSP resources and closes timing at 50 MHz with +1.545 ns worst negative slack. On-board execution reproduced the expected class, checksum, and status, demonstrating a complete and functional software-to-hardware neural-network acceleration workflow.

REFERENCES

- [1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [2] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik, "EMNIST: An extension of MNIST to handwritten letters," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2017, pp. 2921–2926.
- [3] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based accelerator design for deep convolutional neural networks," in *Proc. ACM/SIGDA Int. Symp. FPGA*, 2015, pp. 161–170.
- [4] J. Qiu et al., "Going deeper with embedded FPGA platform for convolutional neural network," in *Proc. ACM/SIGDA Int. Symp. FPGA*, 2016, pp. 26–35.
- [5] Y. Umuroglu et al., "FINN: A framework for fast, scalable binarized neural network inference," in *Proc. ACM/SIGDA Int. Symp. FPGA*, 2017, pp. 65–74.
- [6] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF CVPR*, 2018, pp. 2704–2713.
- [7] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 448–456.
- [8] Xilinx Inc., "Zynq-7000 SoC Technical Reference Manual," UG585, v1.13, 2021.